1 Project Overview

1.1 Introduction and Core Purpose

DefendIt is a Flutter and Firebase mobile application that supports thesis advising between student groups and their advisers. A group submits title ideas, books consultation slots, and tracks its work through six thesis milestones. The adviser accepts groups, reviews titles, records consultation notes and action items, and approves milestones.

1.2 Problem and Objectives

Thesis consultations are often coordinated through chat messages and paper notes. Agreements can be forgotten, feedback becomes scattered, and progress is difficult to follow. DefendIt aims to:

- Let a leader register a group with its section, topic, and two to five members, and let advisers accept up to four groups.
- Let students submit title ideas containing a title, description, and purpose for adviser review.
- Let advisers publish consultation slots that groups can book without double booking.
- Record consultation notes and assignable action items.
- Let groups plan milestone tasks with assignees, due dates, and progress indicators.
- Keep core records available offline in a local database that synchronizes with the cloud.

1.3 Compliance with Project Requirements

Requirement	How DefendIt addresses it
Core purpose	Thesis groups and advisers; scattered feedback, lost agreements, and unclear progress.
Unique UI and UX	Custom teal and amber components with Inter text (Section 4).
Authentication	Google sign-in, roles, sessions, and registration checks (Section 7).
Database	Live Firestore listeners and local SQLite storage (Sections 6 and 8).
Topic exclusivity	Thesis-advising workflow; topic approval is handled through the instructor's approval process.
Mobile application	Android build and historical APK; no fresh build/device verification. CRUD: Section 6.4.
Documentation	Required test coverage is in Section 10.3; a separate test document and Flutter suite are not supplied.

2 Target User Personas

Leaders and teammates share the student role. leaderUid identifies the creator; memberUids grants access. Roster names alone do not register accounts.

Persona	Needs	How DefendIt helps
Group leader	Coordinate a group of up to five, gain title approval, and prepare for meetings.	Title feedback, bookable slots, action items, and milestone tasks.
Group member	Join the group and see assigned work and progress.	Student-number registration, teammate code, and task assignees.
Adviser	Review up to four groups and retain a record of agreements.	Request previews, review tools, notes, and group progress.

3 Requirements

3.1 Functional Requirements

Feature	Required behavior
Sign-in and roles	Google sign-in and a one-time role choice. Advisers claim an access code; students register a unique student number.
Groups	The intended limit is one group per student, checked before create or join. Rosters contain two to five distinct names; joining adds account membership without changing or checking roster capacity. Codes have six characters.
Adviser requests	Preview an open group's name, section, topic, members, and up to three title ideas; accept up to four groups.
Title ideas	Members create proposals and revise permitted ideas. Advisers choose Approved, Needs polish, or Rejected; feedback explains required changes.
Scheduling	Future slots are checked for overlap before creation; this check is not atomic. Booking one slot is transactional. Members or the owner can cancel before its start; booked slots cannot be deleted.
Consultations	Advisers record notes and action items; group members mark action items complete.
Milestones	Six milestones per group. Submission requires at least one task and all tasks done. Assigned advisers approve, request revision, and set dates; tasks lock during review and after approval.
Dashboard	Today shows the next consultation, counts, and a Needs attention list.
Offline records	Notes, action items, title ideas, milestones, and tasks are stored locally and synchronized. A chip displays sync status.

3.2 Non Functional Requirements

Usability requires a consistent theme and distinct loading, empty, error, success, and validation states. Forms should make incomplete or invalid input visible before submission.

Security requires authenticated access and Firestore Security Rules that restrict records according to profile ownership, group membership, and assigned adviser. Client controls and server rules serve different purposes: hiding an action does not replace backend authorization.

Integrity relies on transactions for operations that compete for shared records, including booking and adviser acceptance. The limits and enforcement boundaries are discussed in Section 10.1.

Performance uses SQLite for local records, so already downloaded work can be displayed and edited without waiting for a cloud round trip. Coordination features still access Firestore directly.

Android uses the installed Flutter SDK default minSdk 24 (Android 7.0); the existing release manifest also records API 24. Firebase options exist for Android and web, but web has no SQLite adapter and uses the native Google sign-in path. iOS, macOS, Windows, and Linux options throw UnsupportedError.

4 UI Design and Wireframes

4.1 Visual Identity

The implemented palette is teal #176B63, dark teal #103C3B, pale sage #E3F1ED, amber #E9B65E, paper #F4F7F5, white #FFFFFF, ink #18332F, muted #60756F, border #DCE6E0, success #287B57, and error #C34848. Legacy indigo identifiers now store teal hues. Figures illustrate layouts rather than live screenshots.

The app uses Inter throughout: headings 30/22, titles 16, body 14, captions 12, and buttons 15 logical pixels. This document uses Century Gothic. Login shows DefendIt; MaterialApp.title and Strings.appName still say ThesisTrack.

Cards use a 20-pixel radius, inputs and primary buttons 14, and sheet tops 28 logical pixels. Primary buttons are at least 52 high. Loading, empty, error, and validation states support the workflows.

4.2 Navigation Structure

After Google sign-in and one-time role setup, the user enters a shell with four tabs.

Tab	Student	Adviser
Today	Next consultation, counts, and Needs attention.	The dashboard from the adviser's perspective.
Schedule	Available slots and My bookings.	Availability list and Add slot.
Groups	My group, create or join, and title ideas.	My groups, Requests, previews, and title review.
Progress	Milestones and task screens.	Group progress, approvals, and revision requests.

4.3 Wireframes

Sign-in establishes identity; role setup registers adviser credentials or a student number. Onboarding, then /home, follows the authentication state; the four tabs preserve state in an IndexedStack.

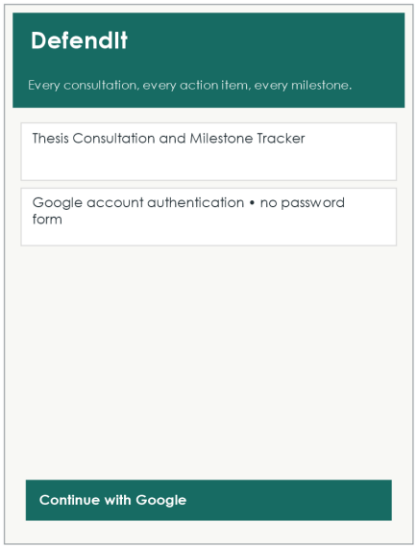


Figure 4.1 Login

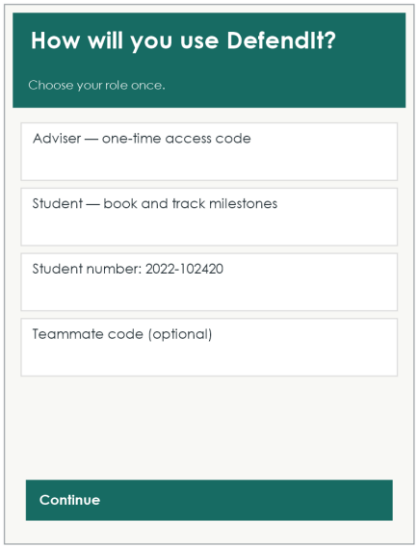


Figure 4.2 Student registration

4.3 Wireframes continued

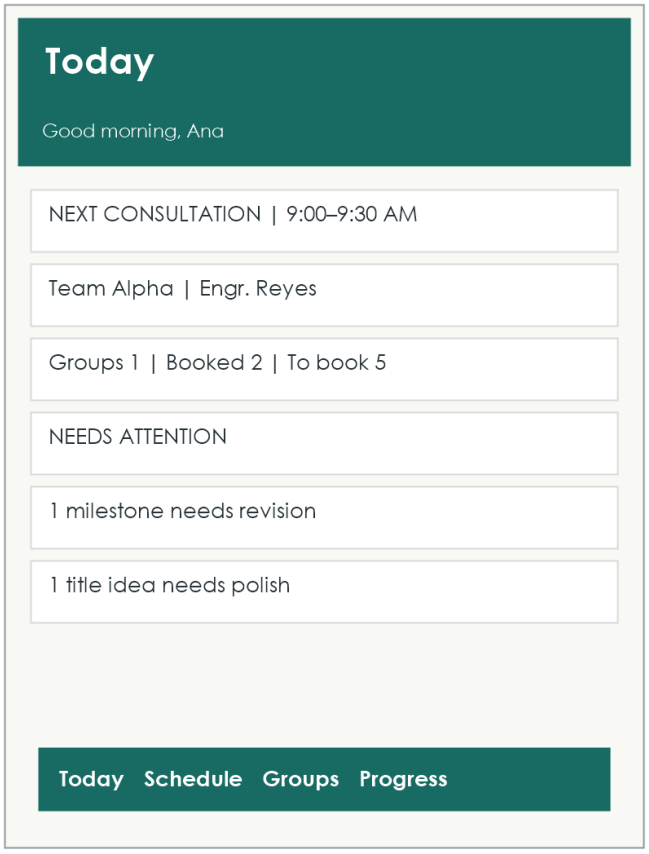


Figure 4.3 Today dashboard

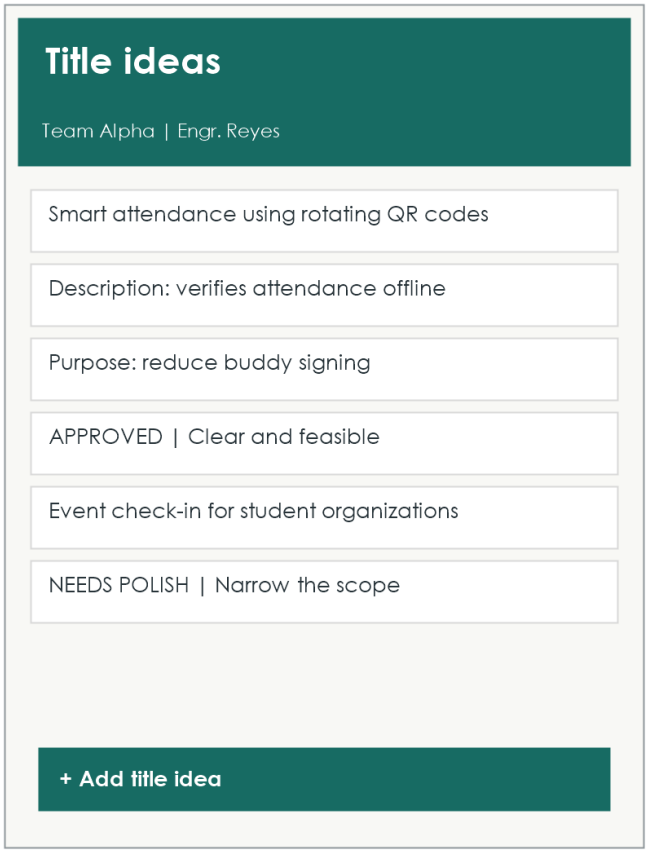


Figure 4.4 Title ideas

Today combines Firestore group and slot streams with a SQLite Needs attention list. Students see revision, polish, overdue, and ready-to-submit items; advisers see pending reviews and overdue work. A pending count is not proof of successful remote delivery.

Title states are pending, approved, needsPolish, and rejected. Members edit pending or needsPolish ideas and delete nonapproved ideas. Editing resets status to pending and clears review fields. Only the assigned adviser reviews; polish and rejection require feedback.

Approved title ideas do not overwrite groups.thesisTitle or automatically approve the title_approval milestone. The example study titles are sample group content.

4.3 Wireframes continued

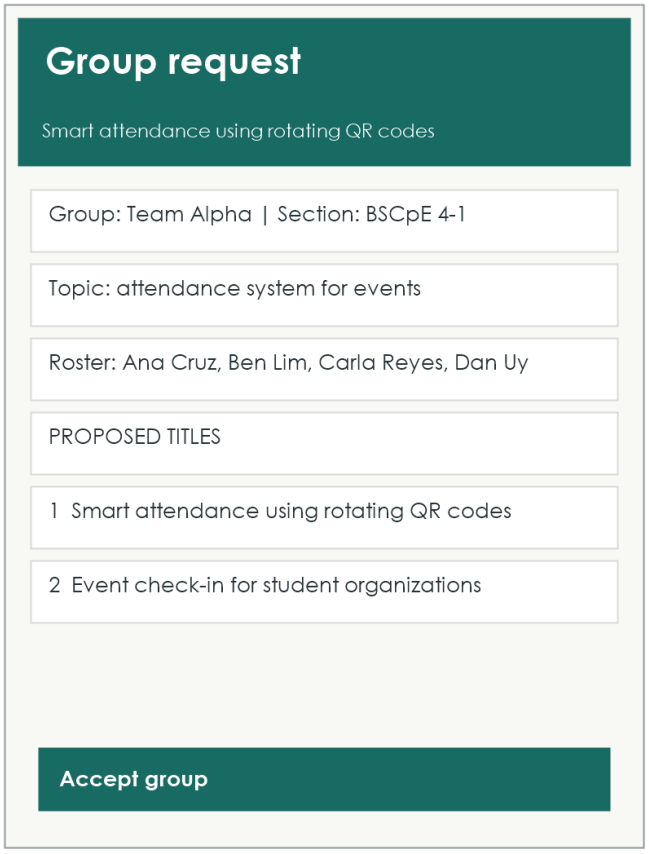


Figure 4.5 Adviser preview

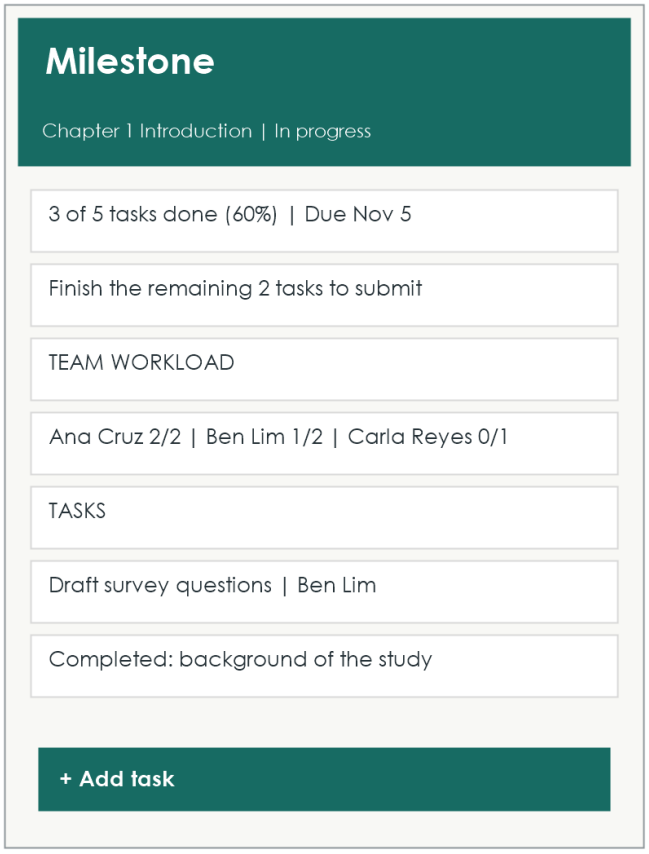


Figure 4.6 Milestone tasks

The preview reads up to three proposals directly from Firestore. Acceptance transactionally sets adviserUid/adviserName and increments users.adviserGroupCount, rejecting an already assigned group or a count of four.

The defaults are Title approval, Chapter 1 Introduction, Chapter 2 Review of Related Literature, Chapter 3 Methodology, Proposal defense, and Final defense. Their positions order the display; they do not enforce prerequisites between milestones.

Students move notStarted to inProgress, then forReview; they may withdraw review or resubmit needsRevision. Advisers move forReview to approved or needsRevision. The first task starts work automatically. Overall progress averages 1 for approved milestones and done/total tasks otherwise, with zero for no tasks.

5 System Architecture

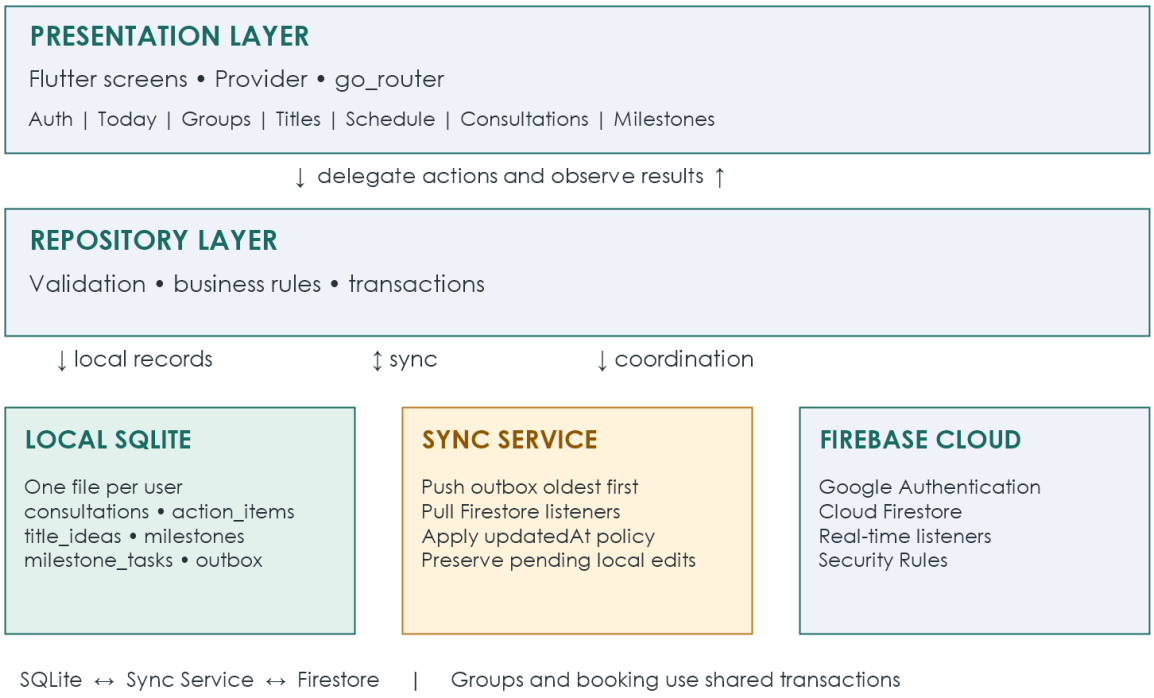


Figure 5.1 System architecture

The presentation layer contains Flutter screens grouped by feature. Provider supplies state, and go_router handles navigation. Screens delegate persistence to repositories rather than reading or writing databases directly.

The repository layer holds feature validation, business rules, and transactions. Authentication, groups, slots, titles, consultations, milestones, and attention summaries each have a repository responsibility.

Offline-first records—notes, action items, titles, milestones, and tasks—are read from SQLite. Local writes save the data change and its outbox entry together. The sync service sends queued operations to Firestore when online and applies incoming listener updates to the local database.

Coordination features, including groups, slots, booking, and authentication, use Firebase directly. Shared-record transactions prevent competing booking or acceptance operations from being treated as independent local edits. Section 8 explains the synchronization boundaries.

The declared Dart constraint is ^3.12.2; pubspec.lock requires Flutter >=3.44.0. Main packages are firebase_core ^4.15.0, firebase_auth ^6.7.0, google_sign_in ^7.2.0, cloud_firestore ^6.10.0, sqflite ^2.4.4+1, provider ^6.1.5+1, go_router ^17.5.0, and connectivity_plus ^7.3.2.

6 Database Schema

6.1 Entity Relationship Diagram



Figure 6.1 Entities fields and relationships

Identifiers connect the entities shown above. A group contains title, milestone, and task subcollections. A consultation reuses its booked slot's identifier, and action items reference their consultation and group. These are stored document references by identifier, rather than relational foreign-key constraints enforced by Firestore.

The diagram follows model payloads and repository writes. Document IDs are separate from most payloads; users also store uid. createdAt on profiles and number claims, and claimedAt on code claims, are Firestore server timestamps. Other date fields are integer epoch milliseconds. Nullable fields are marked ?; lists and maps retain their Firestore types.

6.2 Cloud Firestore Collections

Collection or document path	Purpose
users/{uid}	Profile and role, student number or adviser code, and adviser group count.
studentNumbers/{number}	Reserves a normalized student number for one account.
adviserCodes/{code}	Pre-created adviser access codes and their claim details.
groups/{leaderUid}	Name, section, topic, roster, account membership, join code, and adviser.
groups/{id}/titles/{titleId}	Proposals, review status, feedback, and review metadata.
groups/{id}/milestones/{milestoneId}	Six milestones, their status, dates, and feedback.
groups/{id}/tasks/{taskId}	Tasks, assignee, due date, completion, and completion author.
slots/{slotId}	Adviser availability and booking information.
consultations/{slotId}	Notes and session details for a booked slot.
actionItems/{itemId}	Assigned consultation actions, due date, and completion.

6.3 Local SQLite Database

SQLite version 4 uses thesistrack_{uid}.db with consultations, action_items, title_ideas, milestones, milestone_tasks, and outbox. Mirrored records add id TEXT PRIMARY KEY; done becomes 0/1 and consultation roster becomes JSON TEXT. outbox uses an integer sequence. No SQL foreign keys or cascades are declared. Users, groups, slots, and registration registries have no dedicated SQLite tables.

6.4 CRUD Operations

Entity	Create	Read	Update	Delete
Group	Student once	Members; adviser or request preview	Join; adviser accepts	Not provided
Title idea	Member	Members; adviser or request preview	Pending or Needs polish revision; adviser review	Member if not approved
Milestone	With group	Members; adviser	Work status; review; due date	Not provided
Task	Member	Members; adviser	Member while unlocked	Member while unlocked
Slot	Adviser	Adviser; relevant students	Book; cancel	Owner if open
Consultation	Adviser	Adviser; members	Adviser notes	Not provided
Action item	Adviser	Adviser; members	Completion by members or adviser	Adviser

7 Firebase Configuration and Authentication

Google sign-in uses Firebase Authentication. It avoids the SMS delivery and quota concerns of phone authentication and supports testing with several Google accounts.

7.1 Setup Steps

1. Use the configured project thesis-track-6a918 or an authorized replacement. Register Android package com.example.thesis_track and enable Google authentication.

2. Register the signing key's SHA-1 fingerprint. Run .\gradlew signingReport from the android folder in PowerShell. The source release configuration uses the debug key; register the actual release key if that changes.

3. Run flutterfire configure, place google-services.json in android/app, and set the Web client ID in lib/core/constants/strings.dart.

4. Create Cloud Firestore and configure its Security Rules. This repository has no firestore.rules or rules deployment entry in firebase.json; obtain reviewed rules before publishing. Pre-create issued adviserCodes documents; no fixed count or code-generation tool is implemented.

7.2 Roles Identity and Sessions

Stored roles are adviser and student; leader is group metadata. Adviser onboarding formats an issued code such as CPE-2026-XXXX. Registration uppercases it, removes whitespace, rejects /, and transactionally claims the existing document. Another UID cannot reuse a claimed code; the same UID can.

Student numbers normalize to ten digits with a dash and match ^20\d{2}-\d{6}$. A transaction reserves studentNumbers/{number} for one UID. A teammate code is optional; joining failure after registration leaves the profile saved and shows retry guidance.

Firebase authStateChanges drives loading → /splash, signedOut → /login, needsRole → /onboarding, and ready → /home. Profile-load failure also routes to login. Sessions use Firebase persistence. Shell entry opens the UID database; sign-out stops listeners and closes it without deleting local files.

7.3 Security Rules Summary

Repositories guard title and milestone membership and assigned-adviser actions. Consultation save/add/delete methods have no actor parameter and rely on UI restrictions plus backend rules. Required Firestore policies include ownership, membership, protected identity fields, review permissions, and task locks. No rule file is supplied, so deployed enforcement cannot be confirmed from this repository.

8 Local Database and Synchronization

SQLite saves each local workflow change and its outbox entry in one transaction. outbox fields are seq INTEGER PRIMARY KEY AUTOINCREMENT, collection TEXT, docId TEXT, op TEXT, data TEXT?, and createdAt INTEGER. data is JSON; op is upsert or delete. collection can be a nested group path. The queue is not a Firestore collection.

Flush replays seq order using Firestore merge writes or deletes, removing an entry only after success. Each push has a 12-second timeout. Failures retain the operation and block later entries for that attempt. Retry is triggered by writes, connectivity events, shell startup, or the sync chip; there is no dedicated backoff scheduler.

Listeners pull titles, milestones, and tasks by group. Advisers query consultations and actions by adviserUid; students query groupId. Pending local edits suppress incoming changes for that document. Otherwise a newer local updatedAt blocks an older update; equal timestamps can replace local data. Removed documents are deleted unless locally pending. This is not server-enforced conflict resolution.

Offline edits require previously loaded records. Group and slot coordination, registration, and first consultation creation need internet. Connectivity detection reports network presence, not Firebase reachability. The chip does not display lastError; pending failures may continue to show Syncing. Several listener/apply errors are suppressed.

The four explicit indexes cover action_items.consultationId, title_ideas.groupId, milestones.groupId, and milestone_tasks.milestoneId. Versions 2, 3, and 4 add titles, milestones, and tasks. Sign-out retains each account's file; in-flight sync and account switching require additional race-condition testing.

9 Setup and Installation Instructions

Prerequisites are Flutter SDK, Android Studio and Android SDK, Node.js and Firebase CLI, FlutterFire CLI, and an Android phone or emulator with Google Play. The device must meet the release build's minimum Android version and dependency requirements (Section 3.2).

9.1 Run in Development

1. Clone the repository, open the project in Android Studio, and run flutter pub get from the project root.

2. Complete the Firebase setup in Section 7.1. The package name, Firebase Android registration, OAuth client, and signing fingerprints must refer to the same application configuration.

3. Connect a device with USB debugging enabled, or start an emulator with Google Play services.

4. Run flutter run or use Run in Android Studio. Verify Google sign-in and onboarding before testing shared group records.

9.2 Build the Standalone APK

The main manifest currently labels the app thesis_track and omits INTERNET; debug/profile declare it. The existing merged release manifest includes INTERNET from Google sign-in/Firebase dependencies. Verify the newly merged manifest rather than assuming release networking is absent. Change the label separately if DefendIt must appear in the launcher.

Run flutter clean, then flutter pub get, then flutter build apk --release. Copy build/app/outputs/flutter-apk/app-release.apk to the phone and install it. The installed application runs without a USB connection, laptop, or Android Studio; online features still require internet access.

The source configuration signs release builds with the debug key. Before distribution, use the intended release signing configuration and register its fingerprint with Firebase. Re-test authentication in the generated APK rather than relying only on a debug run.

9.3 Troubleshooting

Problem	Check and corrective action
Google sign-in returns to login	Verify the SHA-1, Google provider, client ID, and package registration. Download matching google-services.json and rebuild.
Debug works but APK sign-in fails	Check INTERNET in the main manifest and the fingerprint of the key that actually signed the APK.
permission-denied	Verify deployed Firestore rules, authenticated identity, saved role, and group membership or adviser assignment.
Adviser code rejected	Confirm that the issued document exists in adviserCodes and is not claimed by another account.
Student number rejected	Check its format and whether another account already registered the number.

10 Limitations Future Improvements and Testing

10.1 Limitations

Conflicts use device clocks. An incorrect clock can make stale work appear newer, and protecting pending local edits does not resolve every disagreement between devices. A queued edit is not evidence that another device has received the same state.

Adviser acceptance checks a shared count in a transaction. Student membership and join-code uniqueness use separate pre-checks, so concurrency can bypass those intended constraints. Joining does not verify roster membership or a five-account capacity. No repository-supplied rules establish backend enforcement. Student numbers remain self-declared.

Groups, slots, and booking require internet access. A consultation must be opened once online before its notes and action items can be used offline. Loaded records and pending operations remain device-specific until synchronization succeeds.

Android is the configured native target; other platform folders are scaffolding. No Flutter test suite or backend integration tests are supplied. Push notifications, group editing, chapter uploads, calendar export, and custom milestone creation/deletion are absent.

10.2 Possible Improvements

- Cloud Functions for reminders and stronger enforcement of group limits.
- A pre-loaded class list to verify student numbers and an editable roster.
- Chapter document uploads and calendar export.

These are possible improvements, not features delivered by the current documentation.

10.3 Testing

Repository verification ran flutter analyze --no-pub: 11 informational findings, no reported errors or warnings, and exit code 1. flutter test --no-pub exited 1 because test/ is missing. iOS/macOS runner tests are empty examples. No device, fresh release-build, or live Firebase authorization test was performed; the table below is required coverage, not passed execution.

Area	Essential verification
Identity	Google sign-in, role registration, adviser-code claims, unique student numbers, persistent sessions, and sign-out.
Group coordination	Create and join a group, preview requests, accept within capacity, and reject conflicting acceptance.
Titles and milestones	Proposal changes and adviser feedback; task completion, submission conditions, approval, and revision locks.
Consultations	Publish and book slots, cancel eligible bookings, save notes, and maintain action items.
Offline synchronization	Keep local edits, retain failed operations, retry when connected, and confirm changes on another account or device.
Access and release	Reject unauthorized writes; install and verify the generated Android APK with its actual Firebase and signing configuration.